

Implementation Report

Reproduction of Proper Scoring Rules for Survival Analysis

Student: Saba Naji

Course: Statistical Learning - Spring 2026

Selected paper: Hiroki Yanagisawa, “Proper Scoring Rules for Survival Analysis” (ICML 2023)

Date: August 2026

1. Paper and Implementation Goal

The paper I implemented is “Proper Scoring Rules for Survival Analysis” by Hiroki Yanagisawa (ICML 2023). The paper studies four scoring rules for right-censored survival data: Portnoy, censored logarithmic score (Cen-log), censored Brier score (Cen-Brier), and censored ranked probability score (Cen-RPS). The main goal of my implementation was to reproduce the central experimental pattern of the paper with public datasets and limited computing resources.

I focused on three questions. First, can the four methods be trained on the same type of neural-network model used in the paper? Second, do Cen-log and Cen-Brier remain the strongest methods, as reported by the paper? Third, are my Cen-log values reasonably close to the exact values reported in Appendix Table 4? I also repeated two smaller studies about the number of time bins and the DeepHit ranking parameter. Finally, I tested one simple extension called Hybrid.

2. Software Environment and Data

I ran the notebook on a CPU-only computer. I used Python 3.14 and PyTorch 2.12.1 with NumPy, pandas, scikit-learn, and matplotlib. The original paper used Python 3.7.4 and PyTorch 1.7.1 on a CPU machine. My software versions are newer, but the general computing setup is similar. I fixed the random seed to 42 so the experiment is easier to repeat.

I used the same three public R datasets named in the paper. The CSV files were exports of flchain from the survival package, prostateSurvival from the asaur package, and support from the casebase package. After preprocessing, the dataset sizes used by my code were:

Dataset	Rows used	Features after preprocessing	Censoring rate	Maximum time
flchain	6,524	8	0.6993	5,166
prostateSurvival	14,294	6	0.7174	119
support	9,104	44	0.3190	2,029

For flchain, I removed rows with missing creatinine and removed the chapter column. For prostateSurvival, I treated status > 0 as an observed death. For support, I removed slos. Categorical variables were converted to dummy variables, missing numeric values were filled with the median, and zero survival times were changed to 0.001. The paper does not describe every preprocessing detail, so this is one possible source of difference between my results and the original results.

The censoring rates match the values given in the paper (69.9%, 71.7% and 31.9%). This was a useful check that I read the correct columns. My flchain has 6,524 rows instead of the 7,874 of the paper because I dropped the rows with a missing creatinine value.

3. Model and Scoring Rules

3.1 Setting and discretization

For each patient we have a feature vector, an observed time, and an indicator. The observed time is the minimum of the true event time and the censoring time, and the indicator says which of the two we saw. The task is to estimate the whole distribution of the event time, not a single number.

A neural network cannot output a continuous function, so the paper discretizes. In distribution regression the time axis is divided into B intervals and the model outputs the probability mass in each interval. In quantile regression the probability axis is divided instead, and the model outputs B increasing quantiles. I used $B = 32$, the value used in the main experiments of the paper.

The difficult part is the censored rows. For an uncensored row we know which interval contains the event, so a normal score can be used. For a censored row we only know that the event happens somewhere after the censoring time. The correct amount of probability mass for each later interval is exactly the weight parameter that the theorems of the paper need, and this weight depends on the true distribution, so it cannot be computed directly. Following the paper, I estimate it from the current model with the iterative reweighting idea of Pearce et al. (2022): at each training step the weight is computed from the present prediction and then treated as a constant. In my code this is the `.detach()` call inside every loss function.

3.2 The four losses

Cen-log (Equation 4) is the censored logarithmic score. An uncensored row contributes the negative log of the probability mass of its own interval. A censored row contributes a weighted mixture of that term and the log of the survival probability after the interval.

Cen-Brier (Equation 8) is the censored Brier score. It minimizes the squared distance between the predicted mass vector and a target vector. For an uncensored row the target is a one-hot vector. For a censored row the remaining mass is spread over the later intervals using the estimated weights.

Cen-RPS (Equation 9) is the censored ranked probability score. It is computed as a sum of binary Brier scores over all interval boundaries.

Portnoy (Equation 2) is the censored pinball loss and works on quantiles instead of masses. For an uncensored row it is the ordinary pinball loss. For a censored row it mixes the pinball loss at the censoring time with a pinball loss at a constant larger than the largest possible time. Here the weight depends on the unknown probability that the event happens before censoring, and I estimate it by counting how many predicted quantiles are below the observed time.

Because Portnoy produces quantiles while the other three produce interval masses, I convert its output into the same interval format before computing any metric. Without this step the five methods cannot be compared on one scale.

3.3 Model architecture and extra objectives

The neural network follows the paper: three hidden layers with 128 neurons each and ReLU activation. For the three distribution-based losses the output layer has 32 units with a softmax, so the predicted masses are positive and sum to one, which makes the estimated distribution function increasing. For

Portnoy the output goes through a positive transformation and a cumulative sum, which keeps the quantiles in the correct order. I used Adam with a learning rate of 0.001.

I also implemented two extra objectives. Cen-log-simple is the simplified form of Equation 5, used as an evaluation score when the number of bins is large. DeepHit adds a ranking term to the simplified log loss. I used DeepHit only in the extra alpha experiment, because the paper argues that the ranking term can hurt proper distribution estimation. DeepHit is also the only loss in my notebook that is not proved to be proper, so it plays the role of the improper baselines of Figure 4 of the paper.

The Hybrid extension uses an equal mixture of the two strongest paper losses:

$$\text{Hybrid loss} = 0.5 \times \text{Cen-log} + 0.5 \times \text{Cen-Brier}$$

This extension does not change the network or add more data. It only changes the training objective. I treated it as an empirical test, not as a new theoretical result.

4. Training and Evaluation Procedure

For each dataset, I used five random train/validation/test splits of 60% / 20% / 20%. Predictors were standardized using the training set only. Every method was trained on the same split so that the comparison was fair. The original paper used 300 epochs. I used 75 epochs because I wanted the full notebook to finish in a reasonable time on a CPU. The validation set was used to keep the best checkpoint: every 25 epochs the validation score is measured, and the weights of the best checkpoint are restored at the end. The paper does not say how it uses its validation split, so this choice is mine.

The main evaluation measures were:

- **Cen-log-simple:** the main discrimination score. It is the only extended rule that can also be used as a metric, because it is the only one that does not depend on an unknown weight. Lower values are better.
- **KM-calibration:** KL divergence between the average predicted survival curve and the Kaplan-Meier curve. Lower values are better.
- **D-calibration deviation:** my simple 20-bin deviation statistic. It is useful for comparing my runs, but it is not exactly the same implementation as the original paper code.

For the smaller experiments on the number of bins and DeepHit alpha, I used three runs instead of five to reduce CPU time.

5. Main Experimental Results

The table below shows the mean results over five random splits. Standard deviations are shown for Cen-log-simple. The main pattern is that Cen-log and Cen-Brier are usually stronger than Cen-RPS and Portnoy. The Hybrid method is competitive, but it does not give a clear improvement on every dataset.

Dataset	Method	Cen-log-simple mean	SD	KM-calibration	D-cal deviation
flchain	Cen-log	1.5581	0.0428	0.0441	0.00002
flchain	Cen-Brier	1.6041	0.0364	0.1531	0.00004

Dataset	Method	Cen-log-simple mean	SD	KM-calibration	D-cal deviation
flchain	Cen-RPS	1.6600	0.0576	0.2043	0.00006
flchain	Portnoy	1.6050	0.0475	0.0902	0.00004
flchain	Hybrid	1.5501	0.0427	0.0430	0.00002
prostateSurvival	Cen-log	1.3760	0.0161	0.0488	0.00001
prostateSurvival	Cen-Brier	1.3754	0.0191	0.0378	0.00001
prostateSurvival	Cen-RPS	1.3866	0.0235	0.1375	0.00001
prostateSurvival	Portnoy	1.3889	0.0231	0.1270	0.00001
prostateSurvival	Hybrid	1.3764	0.0185	0.0360	0.00000
support	Cen-log	1.9051	0.0592	0.0937	0.00024
support	Cen-Brier	1.9553	0.0591	0.1486	0.00026
support	Cen-RPS	2.0422	0.0629	0.4225	0.00032
support	Portnoy	2.0800	0.0739	0.1898	0.00047
support	Hybrid	1.9152	0.0658	0.1084	0.00024

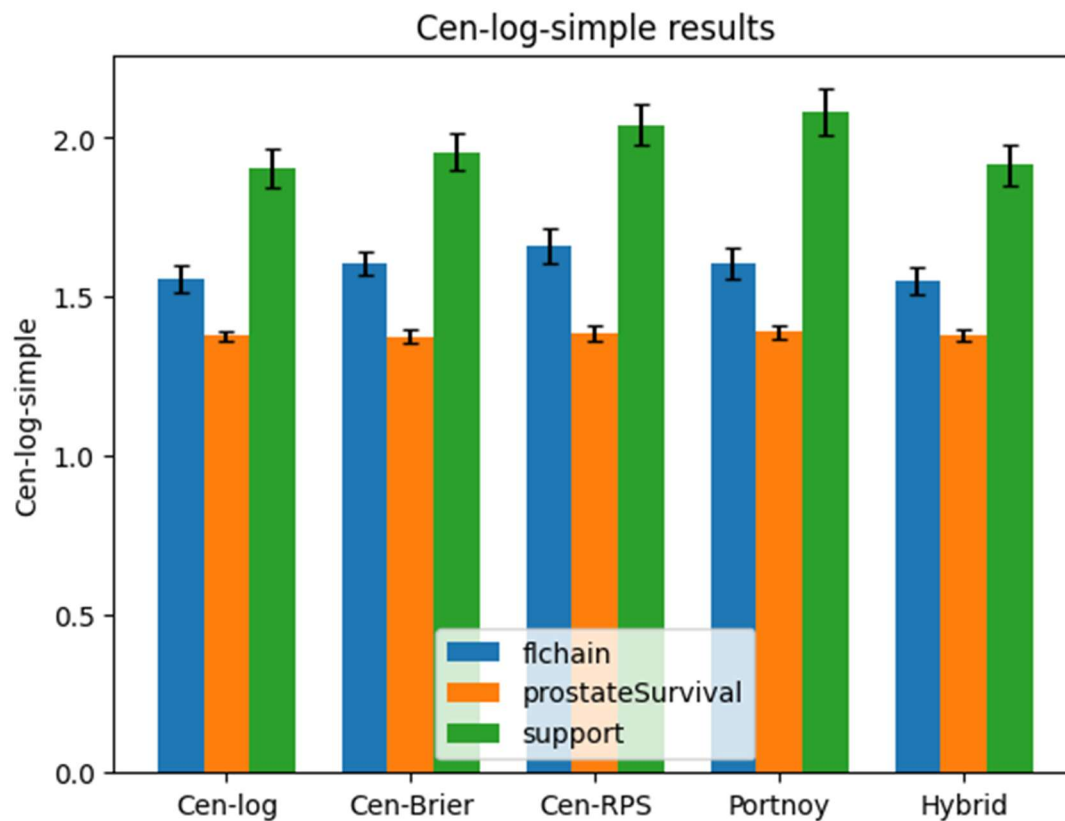


Figure 1. Main Cen-log-simple comparison from the notebook. Lower is better.

For flchain, Hybrid has the lowest mean discrimination score, but Cen-log is very close. For prostateSurvival, Cen-Brier has the lowest mean score. For support, Cen-log is clearly the best of the tested methods. Across the three datasets, Cen-RPS and Portnoy are generally weaker. This is close to the qualitative conclusion of the original paper.

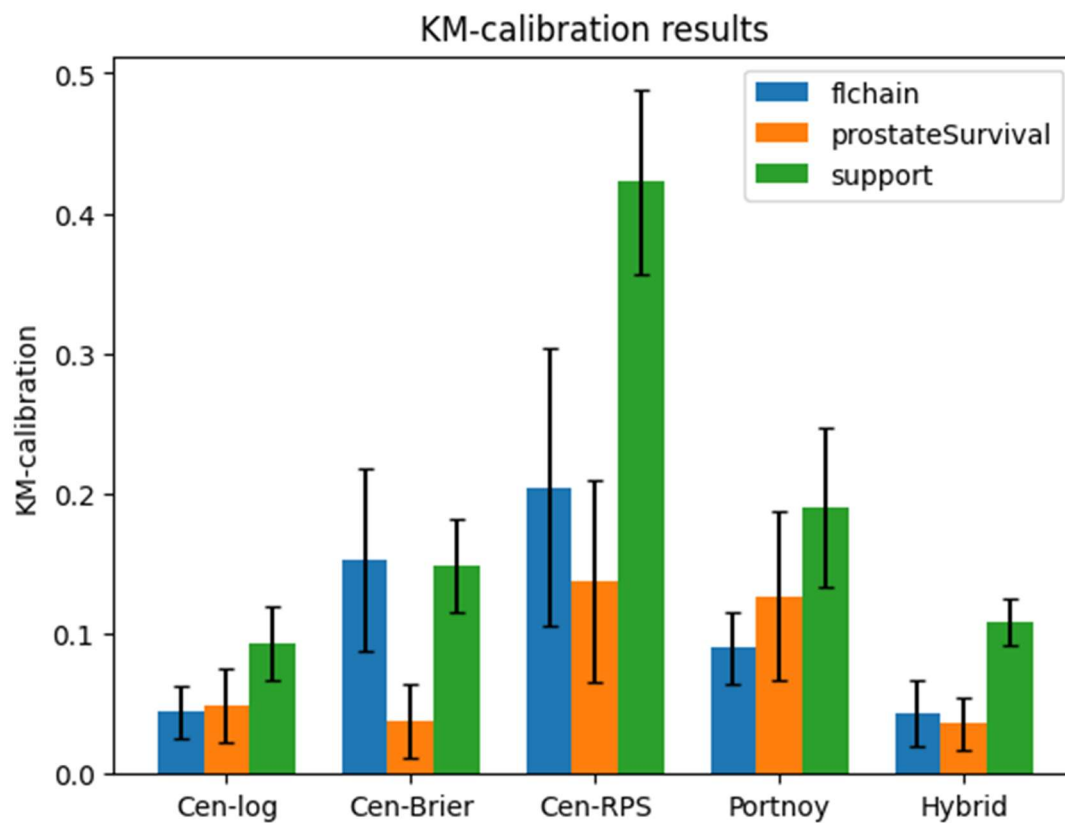


Figure 2. KM-calibration results. Lower is better.

The calibration picture is even clearer than the discrimination picture. On support, Cen-RPS reaches about 0.42 while Cen-log stays near 0.09, so the difference is more than four times. The error bars also show that Cen-RPS and Portnoy are less stable across splits than Cen-log and Cen-Brier. This supports the explanation of the paper: the weights of Cen-log and Cen-Brier are usually close to 0 or 1 and are therefore easier to estimate, while the weights of Cen-RPS and Portnoy can take any value between 0 and 1.

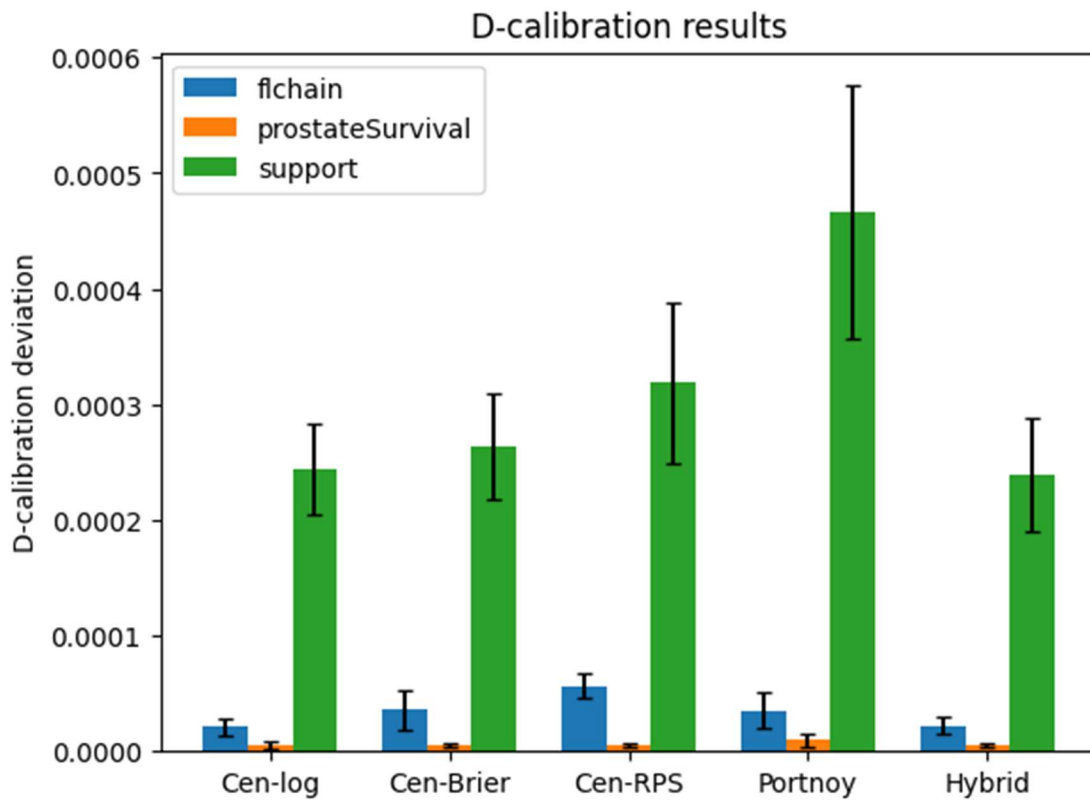


Figure 3. D-calibration deviation. This is my own simple 20-bin statistic.

The D-calibration plot shows the same ordering, but the numbers are very small and this statistic is my own simplified version, so I use it only to compare my own models with each other.

6. Direct Comparison with the Paper

Appendix Table 4 of the paper reports exact Cen-log results for $B = 32$ using Cen-log-simple as the evaluation metric. I compared those values with my five-run Cen-log results:

Dataset	My mean	My SD	Paper mean	Paper SD	Absolute difference
flchain	1.5581	0.0427	1.5054	0.0508	0.0527
prostateSurvival	1.3760	0.0161	1.3608	0.0295	0.0152
support	1.9051	0.0592	1.8307	0.0452	0.0744

The differences are about 0.02 to 0.08. I consider this a reasonable reproduction because the values are in the same range and the ordering is similar. On prostateSurvival the gap is even smaller than the standard deviation reported by the paper itself, so that result is reproduced inside the noise level. On flchain the gap is about the same size as that standard deviation, and on support it is somewhat larger.

One detail is useful here. My values are higher than the published values on all three datasets, and the direction is the same every time. This is what I expect from a shorter training run: with 75 epochs instead of 300 the model is slightly under-fitted, and because a lower score is better, an under-fitted

model gives a higher score. If my differences had different signs on different datasets, I would suspect a mistake in the implementation instead.

The paper reports that the censored logarithmic score and censored Brier score perform best among the four proposed proper scores. My results support this main claim. I did not reproduce every baseline from Figure 4, so this is a partial reproduction of the full benchmark rather than an exact copy of every experiment.

7. Extra Reproduction Studies

7.1 Number of time bins B

The paper compares the exact Cen-log loss of Equation 4 with the simplified Cen-log-simple loss of Equation 5 for $B = 8, 16$, and 32 . The simplified version assumes that the weight is close to zero, which is only reasonable when B is large, so the paper recommends $B > 16$.

B	Dataset	Cen-log score	Cen-log-simple score	Cen-log (KM-cal)	Cen-log-simple (KM-cal)
8	flchain	8.1367	8.0138	0.0034	0.1527
8	prostateSurvival	1.4697	1.4800	0.0061	0.0098
8	support	1.7074	1.7038	0.0038	0.0221
16	flchain	4.4058	4.3850	0.0132	0.0193
16	prostateSurvival	1.3210	1.3230	0.0164	0.0424
16	support	1.6921	1.6975	0.0306	0.0566
32	flchain	1.5611	1.5625	0.0344	0.0237
32	prostateSurvival	1.3856	1.3856	0.0448	0.0478
32	support	1.9054	1.9135	0.0839	0.1430

The discrimination scores of the two objectives are almost identical at every B , so the interesting difference is in calibration. At $B = 8$ the exact version is clearly better calibrated, and on flchain the gap is very large: 0.0034 against 0.1527. As B grows the two versions come together, and at $B = 32$ the difference is small and no longer always in the same direction. This is the behaviour the paper predicts, and it supports the choice of $B = 32$ for the main experiment. Note that the score itself depends on B , so only the two objectives inside the same row should be compared with each other.

7.2 DeepHit ranking parameter

The paper studies DeepHit with ranking weight α . When $\alpha = 0$, the objective is the simplified logarithmic score without the ranking penalty. In my results, performance gets worse as α increases. This gives the same qualitative trend as the paper and supports $\alpha = 0$ when the goal is proper event-time distribution estimation.

Dataset	$\alpha = 0$	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$
flchain	1.5577	1.9258	2.5266	2.6440
prostateSurvival	1.3794	1.4182	1.4826	1.4895

Dataset	alpha = 0	alpha = 0.1	alpha = 1	alpha = 10
support	1.9140	2.2522	2.3549	2.3735

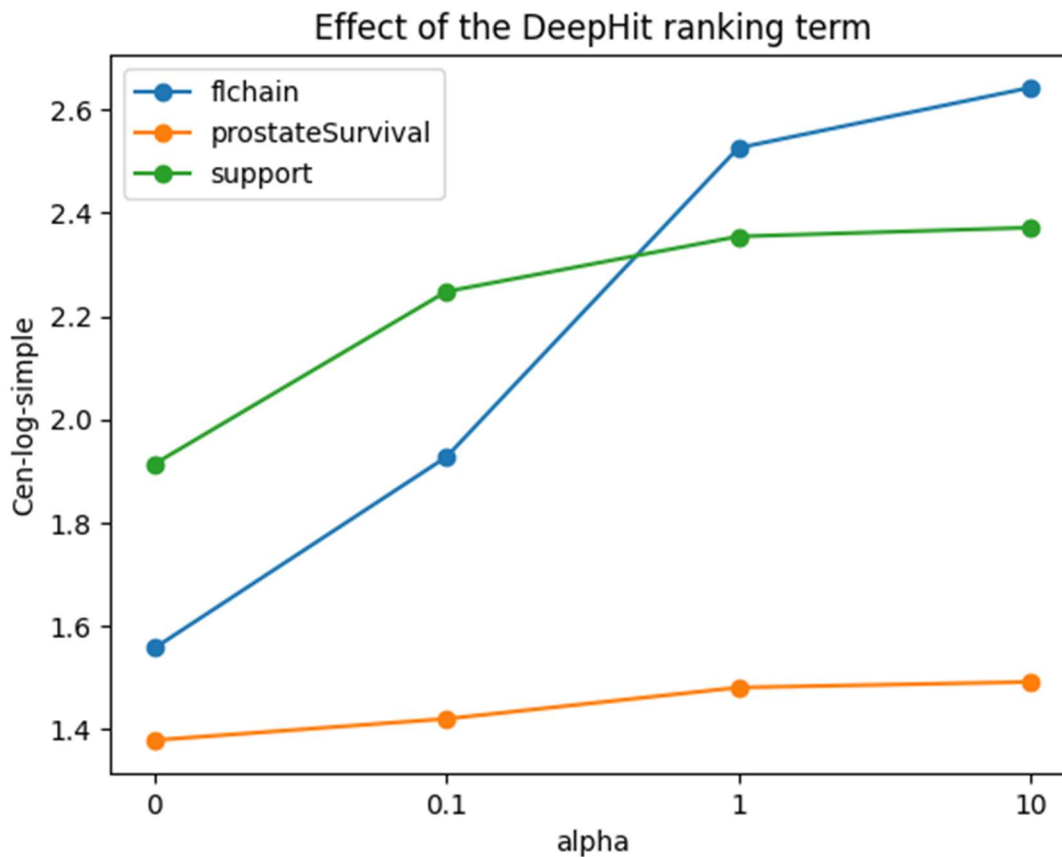


Figure 4. Effect of the DeepHit ranking parameter. Lower Cen-log-simple is better.

The damage to calibration is even stronger than the damage to discrimination. On flchain the KM-calibration goes from 0.0250 at alpha = 0 to 1.6642 at alpha = 10. My values for alpha > 0 are larger than the published ones, which I think comes from my ranking term being computed on a random subset of the training rows to keep the CPU cost down.

8. Extension: Hybrid Loss

8.1 Result

My extension mixes Cen-log and Cen-Brier with equal weight. I chose this idea because these are the two strongest paper methods, and because a positive weighted sum of proper scoring rules is again proper. The extension is easy to test and does not require a larger model. The comparison with the better single method on each dataset is:

Dataset	Hybrid	Best single method	Best single score	Hybrid - best single
flchain	1.5501	Cen-log	1.5581	-0.0081
prostateSurvival	1.3764	Cen-Brier	1.3754	+0.0009
support	1.9152	Cen-log	1.9051	+0.0101

Hybrid improves slightly on flchain, is almost equal on prostateSurvival, and is worse on support. All three differences are much smaller than the standard deviation across splits, so I do not see a clear overall advantage. On calibration the picture is a little better: Hybrid gives the best KM-calibration on flchain and prostateSurvival and the best D-calibration on prostateSurvival and support.

I read this as a negative but informative result. It fits the argument of the paper. Cen-log and Cen-Brier are both proper and both have weights that are usually near 0 or 1, so they are already estimating almost the same thing, and averaging two very similar objectives cannot add much information. If the two rules had failed in different ways, the mixture would have helped more.

8.2 Further extensions I would propose

Learn the mixture weight instead of fixing it. A simple 50/50 mixture may be too restrictive. The weight could be chosen on the validation set, or it could change during training, for example Brier-heavy at the start and log-heavy at the end. This would test whether the squared loss is a better guide in the early epochs, when the estimated weights are still poor. This needs careful validation, because tuning the loss can also overfit the evaluation metric.

Improve the weight estimation itself. Both the paper and my own results point to the unknown weight as the real bottleneck. Instead of taking the weight from the model being trained, it could be taken from a Kaplan-Meier or Cox estimate of the censoring distribution, or averaged over the last few epochs to reduce its variance. Then I would check whether Cen-RPS and Portnoy close the gap to Cen-log and Cen-Brier. If they do, the ordering reported in the paper is mainly about estimation quality and not about the rules themselves, which is a testable version of the explanation given in the paper.

Use non-uniform time bins. I used equal-length intervals, like the paper. Bins based on the quantiles of the Kaplan-Meier estimate would give more resolution in the time region where events actually happen. This should matter most for flchain, where the maximum time is very large compared with the typical event time.

Test properness directly with simulated data. On synthetic data the true distribution is known, so the expected score of the true distribution can be compared with the expected score of deliberately wrong distributions. This would test the theorems of the paper rather than the models, and it would also show how quickly properness breaks when the weight is given a wrong value on purpose.

9. Differences, Limitations, and Final Conclusion

My results should not be expected to match the paper exactly. The most important differences, in my opinion in order of importance, are:

- **Training length.** I used 75 epochs instead of the paper's 300 epochs, which explains why all my values are higher.
- **Undocumented preprocessing.** The paper does not fully specify feature preprocessing, so I used my own transparent pipeline, and my flchain has fewer rows.
- **Undocumented use of the validation split.** The paper reports a 60/20/20 split but does not say what the validation part is used for. I use it for checkpoint selection.
- **Full-batch training.** I did one update per epoch on the whole training set to keep the CPU implementation simple.

- **Simplified D-calibration.** My D-calibration value is a simple deviation statistic and is not the exact statistic from the original code.
- **Portnoy details.** My Portnoy implementation uses a simple grid estimate of the censoring quantile.
- **Missing baselines.** I did not implement the DRSA, S-CRPS, and IPCW baselines from the paper's Figure 4. DeepHit with a positive alpha is the improper baseline in my version.

Even with these differences, the main findings reproduced reasonably well. Cen-log and Cen-Brier were the strongest paper scoring rules in my experiments. The exact Cen-log values were close to the reported values. Larger DeepHit ranking weights made results worse, and the study of B supported using a larger number of time bins. The Hybrid extension did not give a consistent improvement.

Overall, I consider this reproduction successful at the level possible with limited CPU resources. It reproduces the main methods, datasets, model settings, evaluation structure, a direct numerical comparison with the paper, two smaller experimental trends, and one new extension. The remaining differences are explained by clear implementation and resource choices rather than by changing the main research question.

The most useful thing I learned is where the real difficulty of this method is. The proofs of the paper are about properness, but the practical work is about estimating a weight that cannot be observed. My Hybrid extension gave a negative result for the same reason: the two best rules are already almost the same estimator. The more promising extensions are the ones that improve the weight estimation, not the ones that combine the losses.

References

- Haider, H., Hoehn, B., Davis, S., & Greiner, R. (2020). Effective Ways to Build and Evaluate Individual Survival Distributions. *Journal of Machine Learning Research*, 21(85), 1-63.
- Kaplan, E. L., & Meier, P. (1958). Nonparametric Estimation from Incomplete Observations. *Journal of the American Statistical Association*, 53(282), 457-481.
- Lee, C., Zame, W., Yoon, J., & van der Schaar, M. (2018). DeepHit: A Deep Learning Approach to Survival Analysis with Competing Risks. *Proceedings of AAAI*, 32(1), 2314-2321.
- Pearce, T., Jeong, J.-H., Jia, Y., & Zhu, J. (2022). Censored quantile regression neural networks. *Proceedings of NeurIPS 2022*.
- Portnoy, S. (2003). Censored Regression Quantiles. *Journal of the American Statistical Association*, 98(464), 1001-1012.
- Rindt, D., Hu, R., Steinsaltz, D., & Sejdinovic, D. (2022). Survival regression with proper scoring rules and monotonic neural networks. *Proceedings of AISTATS*, PMLR 151, 1190-1205.
- Yanagisawa, H. (2023). Proper Scoring Rules for Survival Analysis. *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, 39165-39182.